Exercises XP: Day 3 - BERT in Practice

Follow the prompts below. Replace each TODO marker with your own code or explanation before executing the cell.

What you'll learn

- How to tokenize text with BERT and understand special tokens.
- How to run a pretrained sentiment pipeline.
- How to build custom BERT-based sentiment and NER analyzers.
- How to compare encoder (BERT) versus decoder (GPT) families.
- How BERT supplies retrieval power inside a RAG stack.

What you will create

- A fully tokenized sentence with visible IDs and special tokens.
- A working sentiment pipeline powered by a fine-tuned DistilBERT model.
- Custom helper classes for sentiment classification and NER.
- A comparison table that contrasts BERT and GPT.
- A written explanation of how BERT embeddings drive retrieval in RAG.

> Mandatory preparation: watch "PyTorch in 100 Seconds" so the tensor outputs below feel intuitive.

Exercise 1 - Tokenization with BERT

Objective: Explore how the bert-base-uncased tokenizer prepares text for model input.

Instructions:

1. (Optional) Install the required libraries.
2. Load the tokenizer, craft a sample sentence, and encode it with padding plus truncation.
3. Print the tokens next to their integer IDs and flag the special tokens.
4. Inspect the attention mask to see how padding is hidden from the model.

Deliverables:

- TODO: Provide the printed list of tokens and IDs with [CLS]/[SEP]/[PAD] highlighted.
- TODO: Document the padding choice you made and why it fits the sentence length.

```
# Optional setup: install dependencies if they are missing in your environme
# %pip install -q transformers torch
```

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

sample_sentence = "hello, I am fatima ezzahra and I am very happy today;"
print(sample_sentence)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: Us
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings ta
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access p
  warnings.warn(
config.json: 100% 570/570 [00:00<00:00, 7.27kB/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 634B/s]
vocab.txt: 232k/? [00:00<00:00, 2.61MB/s]
tokenizer.json: 466k/? [00:00<00:00, 6.23MB/s]
hello, I am fatima ezzahra and I am very happy today;
```

```
encoding = tokenizer(
    sample_sentence,
    add_special_tokens=True,
    padding="max_length",
    truncation=True,
    max_length=24,  # TODO: adjust if your sentence needs more room
    return_attention_mask=True,
    return_tensors="pt"
)

input_ids = encoding["input_ids"][0].tolist()
tokens = tokenizer.convert_ids_to_tokens(input_ids)
print("index | token        | id")
print("-------------------------")
for idx, (token, token_id) in enumerate(zip(tokens, input_ids)):
    print(f"{idx:>5} | {token:<12} | {token_id:>5}")

print("\nAttention mask:", encoding["attention_mask"][0].tolist())
special_positions = [(i, tok) for i, tok in enumerate(tokens) if tok in toke
```

```
print("Special tokens (index, token):", special_positions)
```

```
| token        | id
-------------------
| [CLS]        |   101
| hello        |  7592
| ,            |  1010
| i            |  1045
| am           |  2572
| fatima       | 27596
| e            |  1041
| ##zza        | 20715
| ##hra        | 13492
| and          |  1998
| i            |  1045
| am           |  2572
| very         |  2200
| happy        |  3407
| today        |  2651
| ;            |  1025
| [SEP]        |   102
| [PAD]        |     0
| [PAD]        |     0
| [PAD]        |     0
| [PAD]        |     0
| [PAD]        |     0
| [PAD]        |     0
| [PAD]        |     0

ion mask: [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0
l tokens (index, token): [(0, '[CLS]'), (16, '[SEP]'), (17, '[PAD]'), (18, '[P
```

Exercise 1 reflection

- TODO: Describe how [CLS] and [SEP] behave inside the encoder.
- TODO: Explain how the attention mask hides padded positions from self-attention.

- [CLS] est ajouté au début et sert de résumé global de la séquence. Sa représentation finale est utilisée pour les tâches de classification.
- [SEP] marque la fin d'une phrase ou la séparation entre deux phrases, aidant le modèle à distinguer les segments.

Le masque d'attention met 1 sur les vrais tokens et 0 sur les [PAD] . Ainsi, les positions de padding sont ignorées dans le calcul de l'attention et n'influencent pas la compréhension du modèle.

Résumé simple : [CLS]= résumé, [SEP]= frontière, masque = filtre qui cache le padding.

Exercise 2 - Sentiment analysis pipeline

Objective: Use a pretrained DistilBERT sentiment pipeline to classify a sentence.

Instructions:

1. Import the `pipeline` helper from transformers.
2. Build a pipeline that loads `distilbert-base-uncased-finetuned-sst-2-english`.
3. Pass in a sentence and review the predicted label and score.

Deliverables:

- TODO: Record the sentence you tested.
- TODO: Capture the label plus confidence score and interpret the result.

```
from transformers import pipeline

sentiment_pipeline = pipeline(
    task="sentiment-analysis",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)

sentence = "everything is going well, I am proud of you"
prediction = sentiment_pipeline(sentence)
prediction
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated
config.json: 100%  629/629 [00:00<00:00, 44.5kB/s]
model.safetensors: 100%  268M/268M [00:02<00:00, 111MB/s
Loading weights: 100%  104/104 [00:00<00:00, 185.68it/s, Materializing param=pre_classifier.wei
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 2.16kB/s]
vocab.txt: 232k/? [00:00<00:00, 5.84MB/s]
[{'label': 'POSITIVE', 'score': 0.9998708963394165}]
```

Exercise 2 reflection

- TODO: Does the predicted label match your expectation? Why or why not?
- TODO: How confident is the model and what does the score tell you?

- The model predicted POSITIVE, which aligns perfectly with the intended sentiment.

- The confidence score was 0.99987 (≈ 99.99%). This extremely high probability means the model is almost certain that the sentiment is positive. In practice, such a score indicates that the words going well and proud strongly signal positivity in the training data.

Exercise 3 - Custom sentiment analyzer class

Objective: Rebuild the pipeline manually so you control tokenization, tensors, and scoring.

Instructions:

1. Import `AutoTokenizer` and `AutoModelForSequenceClassification`.
2. Implement `BERTSentimentAnalyzer` with methods for initialization, preprocessing, and prediction.
3. Test the class with multiple sentences.

Hints:

- Keep a `max_length` attribute so you can reuse it while tokenizing.
- Apply `torch.softmax` to transform logits into probabilities.
- Return both the label and the probability for clarity.

```
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from typing import Dict

class BERTSentimentAnalyzer:
    def __init__(self, model_name: str = "distilbert-base-uncased-finetuned-
        '''TODO: load the tokenizer/model and move the model to the proper d
        # Charger le tokenizer et le modèle
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForSequenceClassification.from_pretrained(mode

        # Déterminer le device (GPU si dispo, sinon CPU)
        self.device = torch.device("cuda" if torch.cuda.is_available() else
        self.model.to(self.device)

        # Sauvegarder la longueur max
        self.max_length = max_length


    def preprocess(self, text: str) -> Dict[str, torch.Tensor]:
        '''TODO: clean the text, tokenize, and return tensors ready for infe
        return self.tokenizer(
            text,
            add_special_tokens=True,
            padding="max_length",
            truncation=True,
            max_length=self.max_length,
```

```
            return_tensors="pt"
        ).to(self.device)




    def predict(self, text: str) -> Dict[str, float]:
        # Prétraitement
        inputs = self.preprocess(text)

        # Inférence
        with torch.no_grad():
            outputs = self.model(**inputs)

        # Logits -> probabilités
        probs = torch.softmax(outputs.logits, dim=-1).cpu().numpy()[0]

        # Récupérer label et probabilité
        label_id = probs.argmax()
        label = self.model.config.id2label[label_id]
        confidence = float(probs[label_id])

        return {"label": label, "probability": confidence}
```

```
# TODO: instantiate your analyzer and test several sentences once the class
analyzer = BERTSentimentAnalyzer()
samples = [
    "I feel wonderful today, everything is perfect!",
    "This is a terrible mistake, I am very upset."
]


for text in samples:
    print(text)
    print(analyzer.predict(text))
```

```
Loading weights: 100%   104/104 [00:00<00:00, 468.28it/s, Materializing param=pre_classifier.wei

I feel wonderful today, everything is perfect!
{'label': 'POSITIVE', 'probability': 0.999870777130127}
This is a terrible mistake, I am very upset.
{'label': 'NEGATIVE', 'probability': 0.9996633529663086}
```

Exercise 4 - BERT for Named Entity Recognition

Objective: Build a lightweight class that runs a token-classification model and maps tokens to entity labels.

Instructions:

1. Import `AutoTokenizer` and `AutoModelForTokenClassification`.
2. Implement `BERTNamedEntityRecognizer` with init plus a `recognize` method.
3. Tokenize sample text, run the model, convert the predictions to entity spans, and test with a short paragraph.

Deliverables:

- TODO: Return a list of dictionaries like `{text, entity, start, end}` for each detected entity.
- TODO: Explain how you handled subword tokens that begin with `##`.

```
from transformers import AutoTokenizer, AutoModelForTokenClassification
import torch

class BERTNamedEntityRecognizer:
    def __init__(self, model_name: str = "dslim/bert-base-NER"):
        # Charger le tokenizer et le modèle
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        self.model = AutoModelForTokenClassification.from_pretrained(model_n

        # Déterminer le device
        self.device = torch.device("cuda" if torch.cuda.is_available() else
        self.model.to(self.device)


    def recognize(self, text: str):
        # Tokenisation
        inputs = self.tokenizer(
            text,
            return_tensors="pt",
            truncation=True,
            is_split_into_words=False
        ).to(self.device)

        # Inférence
        with torch.no_grad():
            outputs = self.model(**inputs)

        logits = outputs.logits
        predictions = torch.argmax(logits, dim=-1)[0].cpu().numpy()
        tokens = self.tokenizer.convert_ids_to_tokens(inputs["input_ids"][0]

        entities = []
        current_entity = None

        for idx, (token, pred_id) in enumerate(zip(tokens, predictions)):
            label = self.model.config.id2label[pred_id]

            if label.startswith("B-"):  # début d'entité
                if current_entity:
                    entities.append(current_entity)
                current_entity = {
                    "text": token.replace("##", ""),
```

```
                        "entity": label[2:],  # enlever le préfixe B-
                        "start": idx,
                        "end": idx
                    }
                elif label.startswith("I-") and current_entity:
                    # continuation de l'entité
                    current_entity["text"] += token.replace("##", "")
                    current_entity["end"] = idx
                else:
                    if current_entity:
                        entities.append(current_entity)
                        current_entity = None

            if current_entity:
                entities.append(current_entity)

            return entities
```

```
# TODO: instantiate the recognizer and test it on text that includes people,
ner = BERTNamedEntityRecognizer()
sample_text = "Fatima joined Oracle in Casablanca to work on AI projects."


results = ner.recognize(sample_text)

for ent in results:
    print(ent)
```

```
Loading weights: 100%   199/199 [00:00<00:00, 636.47it/s, Materializing param=classifier.weight]
BertForTokenClassification LOAD REPORT from: dslim/bert-base-NER
Key                       | Status     |  |
--------------------------+------------+--+-
bert.pooler.dense.bias    | UNEXPECTED |  |
bert.pooler.dense.weight  | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture
{'text': 'Fat', 'entity': 'PER', 'start': 1, 'end': 1}
{'text': 'ima', 'entity': 'PER', 'start': 2, 'end': 2}
{'text': 'Oracle', 'entity': 'ORG', 'start': 4, 'end': 4}
{'text': 'Casablanca', 'entity': 'LOC', 'start': 6, 'end': 6}
{'text': 'AI', 'entity': 'MISC', 'start': 10, 'end': 10}
```

Exercise 5 - Comparing BERT and GPT

Objective: Summarize how encoder-style models differ from decoder-style models.

Fill the table with concise statements (one line each).

Category	BERT	
Architecture	Encodeur bidirectionnel basé sur Transformer	Decodeur unidirectionnel (a
Primary purpose	Compréhension du langage (analyser, encoder le sens)	Génération de texte (prédire
Typical use cases	Classification, NER, QA extractive, recherche sémantique	Chatbots, rédaction créative
Strengths	Excellente représentation contextuelle, robuste pour tâches NLP	Fluide en génération, flexible
Weaknesses	Ne génère pas de texte librement, limité aux tâches supervisées	Moins précis pour compréhe

- En bref : BERT est conçu pour comprendre le langage, tandis que GPT est conçu pour produire du langage.

Exercise 6 - BERT inside Retrieval-Augmented Generation

Objective: Explain how BERT-generated embeddings power the retrieval stage of a RAG workflow.

Address each bullet with a short paragraph:

1. TODO: Describe how BERT encodes queries and documents.
2. TODO: Explain how those embeddings are stored and searched in a vector database.
3. TODO: Outline how the retrieved passages are handed to a generative model like GPT.
4. TODO: Provide a concrete application example (industry or product) where RAG with BERT makes sense.

1. Comment BERT encode les requêtes et documents

BERT prend une phrase (requête ou document) et la transforme en un vecteur dense (embedding) qui capture son sens contextuel. Grâce à son encodage bidirectionnel, il comprend les relations entre les mots et produit une représentation riche du contenu.

2. Stockage et recherche dans une base vectorielle

Ces embeddings sont stockés dans une base vectorielle (comme FAISS, Pinecone ou Milvus). Lorsqu'une requête arrive, son embedding est comparé à ceux des documents via une mesure de similarité (souvent la distance cosinus). Les passages les plus proches sont rapidement retrouvés.

3. Passage à un modèle génératif (GPT)

Les documents les plus pertinents sont ensuite fournis comme contexte au modèle génératif (ex. GPT). GPT utilise ces passages pour produire une réponse cohérente et informée, en combinant sa capacité de génération avec les informations récupérées.

4. Exemple concret d'application

Dans le domaine juridique, un cabinet peut utiliser RAG avec BERT pour encoder des milliers de décisions de justice. Lorsqu'un avocat pose une question, les passages pertinents sont retrouvés et transmis à GPT, qui génère une synthèse claire et contextualisée. Cela permet de gagner du temps et d'améliorer la précision des recherches.